

Does Löb’s rule shorten proofs?

MAIS-O11: consolidated progress and Lean certificates

Research record for Andrew Hagy

AI-assisted mathematical development and verification audit

23 September 2026

Abstract

This report consolidates the slow-verifier and connective-tag constructions, the conditional quantitative Löb conversion, the raw-string quotation and trace compiler, the proof-assembly obstruction, and the latest finite-budget envelope theorem. The strongest new structural conclusion is exact: for a fixed provability predicate and toll, absence of a part 1 witness is equivalent to sublinearity of the maximum excess over the reflection-proof length. This negative condition implies a joint linear bound for the overhead function. Conversely, failure of every joint linear bound already yields witnesses with every fixed integer factor. These implications are verified in Lean.

Neither part has been decided for the intended ordinary PA_{bin} checker. The supplied certificates establish the precise abstract, syntactic, and numerical statements identified below; they do not instantiate every arithmetical hypothesis of the earlier chosen-presentation constructions. The accompanying cumulative archive preserves six Lean projects, their research notes, and a single command for rebuilding and auditing all six.

How to read this report. Sections 2–3 give the latest structural result and its full mathematical proof. Sections 4–7 record the earlier mechanisms, implementation milestones, and failed transfer routes. Section 9 maps the claims to certificates and explains how to verify them locally. The report supersedes the earlier manuscript as the summary of *verification status*; the earlier manuscript and detailed milestone notes remain in the archive as research history.

Contents

1	The fixed question and the scope of the comparison	3
1.1	A necessary finite-proof convention	3
1.2	What is being compared	3
1.3	The intended PA-bin presentation	3
2	The finite-budget envelope: an exact characterization	4
2.1	Finite files supply the interface	5
2.2	Computability and proof-search time	5
3	Consequences linking parts 1 and 2	5
4	Earlier chosen-presentation constructions	6
4.1	Finite diagonal hardness and the first-pass certificate	6
4.2	The nullary-predicate slow verifier	7
4.3	Connective tags repair the substitution defect	7
5	The conditional polynomial upper-bound route	8
6	Raw-string quotation and compact arithmetic traces	9
6.1	A compiler that avoids printing the expanded numeral	9
6.2	Local equality certificates	9
6.3	A single coded witness for the arithmetic trace	10
6.4	Checked examples and their interpretation	10
7	Proof assembly and the finite-diagonal obstruction	10
7.1	A concrete suffix with charged binary references	11
7.2	Why the original finite Gödel family triggers the issue	11
7.3	A stronger-consistency family and the remaining gap	12
7.4	Why no universal cheap-box shortcut is available	12
8	What remains for ordinary PA-bin	13
9	Certificate ledger and local reproduction	13
9.1	What the cumulative archive contains	13
9.2	Run the certificates on a Mac	14
9.3	How to interpret the audit	15
9.4	Compile this report	15
	References	16

1 The fixed question and the scope of the comparison

Fix one proof system S , one arithmetization $\Box_S$ of its checker, and one nonnegative integer $c = C_1$ for the cheap-reflection toll. For a provable sentence P , write

$$d(P) = \ell_S(P), \quad r(P) = \ell_S(\Box_S P \rightarrow P), \quad s(P) = |P|, \quad t(P) = c(s(P) + 1). \quad (1)$$

The agenda's cheap direction is $r(P) \leq d(P) + t(P)$. Part 1 asks for a fixed $\delta > 0$ and a sequence P_i such that

$$r(P_i) \rightarrow \infty, \quad d(P_i) \geq (1 + \delta)r(P_i) + t(P_i). \quad (2)$$

Part 2 concerns the fixed ordinary PA_{bin} presentation and the function

$$F_S(k, n) = \max(\{d(P) : r(P) \leq k, s(P) \leq n\} \cup \{0\}). \quad (3)$$

It asks whether this function has a fixed polynomial upper bound in $k + n$. Adding 1 to this argument is harmless for growth; the ordinary checker accepts no empty proof, so the zero-budget corner can also be handled directly. The maximum includes 0 to cover an empty set of eligible sentences.

1.1 A necessary finite-proof convention

The agenda allows $\ell_S(P) = \infty$ for unprovable P . If the displayed part 1 condition is read literally in the extended naturals, without requiring finite proofs, it has an unintended loophole. In a consistent system satisfying Löb's theorem, both $\perp$ and $\Box_S \perp \rightarrow \perp$ are unprovable. The constant sequence $P_i = \perp$ then has $d(P_i) = r(P_i) = \infty$ and satisfies an extended-arithmetic reading of the inequality and divergence. This is not proof shortening. Throughout this report, part 1 concerns *provable* sentences and finite minimum lengths. Löb's theorem ensures that a sentence with a reflection proof has a direct proof; the cheap direction gives the converse for the families under consideration.

1.2 What is being compared

A shortest completed S -proof of P may itself use a formalized Löb argument. Thus $d(P)$ already minimizes over proofs obtained that way. The quantity $r(P)$ measures the premise *before* the cost of converting it into an S -proof of P has been paid. One can equivalently view the reflection file as a new certificate format with an external Löb-rule step about the old box. Changing $\Box_S$ changes the question even when the two checkers accept the same standard files.

The headline question about saving one symbol is weaker than (2). Unbounded additive savings alone need not produce a fixed fractional saving after the toll. For example, the numerical pattern $d = r + \lfloor \sqrt{r} \rfloor$, with zero toll, has unbounded additive excess but no fixed-factor family. This is an illustration of the quantifiers, not an asserted family of PA sentences.

1.3 The intended PA-bin presentation

The uploaded MAIS-A1 fixes a Hilbert presentation based on Enderton, binary numerals, a finite character alphabet, tagged proof records, and priced nested abbreviations. Definitions may name *arbitrary strings*; later fragments may reference earlier definitions. The checker expands these definitions and checks the resulting proof. Hence a short source file may expand enormously. The specified logical axiom class includes all propositional tautologies, which must not silently be replaced by a fixed finite Frege basis.

Two distinctions matter for the remaining work. A polynomial bound in expanded length need not be polynomial in written file length. Also, a Lean proof that a compiler preserves values is

not an object-level PA proof of the fixed arithmetized checker predicate. These issues motivate the quotation work in Section 6.

2 The finite-budget envelope: an exact characterization

This section assumes only finite-valued d, r, s on the target set and that there are finitely many targets with $r(P) \leq k$ for each k . In a proof system over a finite alphabet, that finiteness follows because there are finitely many files of bounded length and each file has a single conclusion. It does not require a bound on the length of an expanded conclusion.

Definition 2.1 (Excess and envelope). Set

$$e(P) = \max\{d(P) - r(P) - t(P), 0\}, \quad H(k) = \max(\{e(P) : r(P) \leq k\} \cup \{0\}). \quad (4)$$

The envelope H is nondecreasing and finite, and

$$d(P) \leq r(P) + t(P) + H(r(P)). \quad (5)$$

A natural-valued function h is called sublinear when

$$\forall q \in \mathbb{N}_{>0} \exists K \forall k \geq K : \quad qh(k) < k. \quad (6)$$

This is the usual statement $h(k) = o(k)$, written without real division.

It suffices to consider factors $1 + 1/q$: for any $\delta > 0$, choose an integer $q \geq 1$ with $1/q \leq \delta$. For $r(P) > 0$, the separation with this factor is equivalent to

$$(q + 1)r(P) + qt(P) \leq qd(P) \iff r(P) \leq qe(P). \quad (7)$$

Indeed, separation forces $d(P) > r(P) + t(P)$, after which the equivalence follows by subtraction. If $d(P) \leq r(P) + t(P)$, both sides are false.

Theorem 2.2 (Exact negative condition). *The following conditions are equivalent.*

- (i) *There is no family satisfying (2).*
- (ii) $H(k) = o(k)$.
- (iii) *There is a function $h = o(k)$ such that, for every target P ,*

$$d(P) \leq r(P) + t(P) + h(r(P)).$$

Proof. First rewrite (i) as the cutoff condition

$$\forall q \geq 1 \exists K \forall P \quad r(P) \geq K \implies qe(P) < r(P). \quad (8)$$

If this condition fails for some q , there are witnesses at arbitrarily large positive reflection lengths by (7). Choose one with $r(P_i) \geq i + 1$ for each i . This gives the required divergent sequence. Conversely any separating sequence eventually exceeds a proposed cutoff. Using $K + 1$ where necessary avoids the exceptional case $r(P) = 0$.

Suppose (8) holds and fix q . Choose its cutoff K and take $k \geq qH(K) + 1$. If $H(k) = 0$, then $qH(k) < k$. Otherwise, choose an attaining target P with $r(P) \leq k$ and $e(P) = H(k)$. If $r(P) \geq K$, the cutoff gives $qH(k) = qe(P) < r(P) \leq k$. If $r(P) < K$, then $e(P) \leq H(K)$ and $qH(k) \leq qH(K) < k$. Thus $H = o(k)$.

Conversely, if $H = o(k)$, choose a cutoff with $qH(k) < k$. For any target whose reflection length exceeds that cutoff, $qe(P) \leq qH(r(P)) < r(P)$, proving (8). This establishes (i) $\Leftrightarrow$ (ii).

Condition (ii) implies (iii) by taking $h = H$ in (5). If (iii) holds, then $e(P) \leq h(r(P))$. The sublinearity cutoff for h immediately gives (8), proving (iii) $\Rightarrow$ (i). $\square$

Verification scope. The equivalences, cutoff formulation, and witness-sequence selection are proved in `Envelope.lean`. Earlier notes treated a sublinear additive converter bound as a stronger sufficient negative condition. The theorem above improves that statement: in the finite-budget setting it is equivalent to the negative answer. It does not prove that the condition holds for PA-bin.

2.1 Finite files supply the interface

The module `FiniteFiles.lean` enumerates all words of length at most k over $\text{Fin}(a)$, decodes them, and collects successful targets. Its minimum-length premise is precisely

$$r(P) \leq k \iff \exists w (|w| \leq k \wedge \text{decode}(w) = \text{some}(P)). \quad (9)$$

The resulting finite target list supplies the hypothesis used in Theorem 2.2. The decoder is a parameter, not an implemented PA parser. Equation (9) is the defining property of minimum reflection length; it is not a hidden polynomial-internalization assumption.

2.2 Computability and proof-search time

For a decidable checker with recognizable reflection conclusions, H is total computable as a metamathematical matter. Enumerate accepted reflection files of length at most k , obtaining finitely many targets and their minimum reflection lengths. For each target, search ordinary proofs in increasing length until a direct proof is found. Löb's theorem guarantees termination. Compute the maximum excess.

There is no useful runtime estimate in this argument. Similarly, shortest direct-proof search converts an accepted reflection file to an ordinary proof; if H is sublinear, its output length satisfies the envelope bound. This does not give a fast conversion algorithm. The current Lean project verifies the finite-list and numerical results, not the computability argument or a PA proof-search implementation.

3 Consequences linking parts 1 and 2

Theorem 3.1 (Near-unit global bounds). *If there is no part 1 witness, then for each integer $q \geq 1$ there is a constant B_q such that, for every target,*

$$qd(P) \leq (q+1)r(P) + qt(P) + B_q. \quad (10)$$

In particular, for some constant B ,

$$F_S(k, n) \leq 2k + c(n+1) + B \leq (2+c+B)(k+n+1). \quad (11)$$

Proof. Choose K from (8) and put $B_q = qH(K)$. If $r(P) \geq K$, then $qe(P) < r(P)$. If $r(P) < K$, then $qe(P) \leq qH(K)$. In both cases,

$$qe(P) \leq r(P) + qH(K).$$

Multiply $d(P) \leq r(P) + t(P) + e(P)$ by q and substitute this estimate to obtain (10). Set $q = 1$ and maximize over $r(P) \leq k$, $s(P) \leq n$ to get the first inequality in (11); elementary arithmetic gives the second. $\square$

Theorem 3.2 (Nonlinearity gives every fixed factor). *Suppose there is no constant C such that $F_S(k, n) \leq C(k + n + 1)$ for all k, n . Then, for every integer $a \geq 1$ and every cutoff K , there is a target P with*

$$r(P) \geq K, \quad d(P) \geq ar(P) + c(s(P) + 1). \quad (12)$$

For every $a \geq 2$ this supplies a part 1 family with $\delta = a - 1$.

Proof. Suppose (12) fails for particular a, K . For all targets with $r(P) \geq K$ we have $d(P) < ar(P) + t(P)$. For a target with $r(P) < K$, (5) and $a \geq 1$ give

$$d(P) \leq r(P) + t(P) + H(K) \leq ar(P) + t(P) + H(K).$$

Hence this last bound holds for every target. Maximizing gives

$$F_S(k, n) \leq ak + c(n + 1) + H(K) \leq (a + c + H(K))(k + n + 1),$$

contrary to the hypothesis. Select witnesses with $r(P_i) \geq i + 1$ to obtain divergence. $\square$

These theorems refine the research alternatives. A negative answer to part 1 would imply the positive answer to part 2 in the stronger linear form. A superpolynomial lower bound for part 2 would imply part 1; even failure of all joint linear bounds suffices. The reverse implication is not established: a fixed-factor gap can coexist with a linear bound having a larger constant. Thus proving a polynomial upper bound alone would not rule out part 1.

Verification scope. The integer statements and implications in this section are proved in `Envelope.lean`. Their hypotheses are not proved for the actual PA-bin length functions. The result narrows the alternatives without selecting one of them.

4 Earlier chosen-presentation constructions

The first attempts deliberately used the freedom to choose S in part 1. Their essential mechanism was a slow finite test built into the checker. This section records the mathematical constructions and their limitations; the archive retains the complete earlier manuscript and tagged-checker note.

4.1 Finite diagonal hardness and the first-pass certificate

For a consistent ordinary base calculus B , a parameterized diagonal family can express

$$D_n \longleftrightarrow \text{there is no } B\text{-proof of } D_n \text{ of written length at most } n.$$

If a short proof existed, numeralwise verification of that particular proof would yield the negation of the displayed assertion, while the proof of D_n and the diagonal equivalence yield the assertion itself. Consistency therefore gives $\ell_B(D_n) > n$. The bounded search actually terminates with a positive result, and PA can verify that finite computation, so each standard D_n is provable. This says nothing favorable about the length of that verification.

The first-pass project formalizes the abstract contradiction, Löb's rule from explicit derivability and diagonal hypotheses, existence of shortest proofs, and a quantitative transfer. In the transfer, a linear interpretation of source proofs gives $n < C(d_n + 1)$, while

$$r_n \leq a(\lfloor \log_2 n \rfloor + 2), \quad s_n + 1 \leq b(\lfloor \log_2 n \rfloor + 2)$$

supply a factor-two subsequence, including any fixed integer toll. Finitely many bounded-length files and distinct reflection conclusions prove that the minimum reflection lengths diverge.

Verification scope. The theorem `SeparationHypotheses.conditional_main` is conditional. No value of its complete hypothesis structure for PA or the guarded verifier has been constructed. Its axiom audit does not remove those hypotheses.

4.2 The nullary-predicate slow verifier

The earlier manuscript adds primitive nullary predicates p_n , whose binary names have logarithmic length. It allows the reflection axiom $\Box_S p_n \rightarrow p_n$ and checks, whenever p_n occurs, the finite condition that D_n has no short B -proof. A parameterized diagonal construction of the checker formula aligns $\Box_S$ with this actual checking procedure, avoiding an informal circular definition.

The interpretation $p_n \mapsto D_n$ takes the reflection axioms to instances of a fixed base-theory theorem supplied by the guard. With whole-expression abbreviations and the specified proof-record convention, the manuscript gives a linear written-length translation. It derives, under consistency of B ,

$$|p_n| + 1 = \mathcal{O}(\log(n + 2)), \quad r_S(p_n) = \mathcal{O}(\log(n + 2)), \quad d_S(p_n) > n/C - 1.$$

This is a proposed affirmative construction for the literal chosen-system clause, with a slow checker and an expanded signature. It is not the fixed PA-bin presentation. The actual checker arithmetization and translation have not been implemented end to end in Lean.

An arithmetic-only variant used a recognizable family G_n and a guard depending on the numeral instance. Although its accepted standard files agree with the ordinary checker's files under consistency, it fails a stronger uniform instantiation property: for $\tau(x) = G(x) \rightarrow G(x)$, a uniform theorem asserting $\Box_* \tau(\bar{n})$ would imply all the hard guards at once. That variant therefore cannot supply the agenda's additional uniform bounded principles. This failure motivated connective tags.

4.3 Connective tags repair the substitution defect

Choose an ordinary finite-basis PA calculus B with primitive conjunction and whole-expression abbreviations. Let

$$T_0 = (0 = 0), \quad T_{n+1} = (0 = 0) \ \& \ T_n, \quad P_n = T_n \ \& \ D(\bar{n}),$$

where the diagonal construction is performed for the tagged formula itself, and $D(n)$ asserts that P_n has no B -proof of length at most 2^n . Guard indices are read from conjunction shapes, ignoring the terms in atomic equalities. Capture-avoiding term substitution preserves these shapes and cannot manufacture a new guard. MP, generalization, and universal instantiation preserve the required guard conditions.

The guarded checker S runs the ordinary check and all the finite guard tests. Consistency gives truth of every standard guard, so the accepted standard files, conclusions, and written lengths agree with B . Their arithmetic provability predicates nevertheless differ. The guard supplies the implication $\Box_S P_n \rightarrow D(\bar{n})$; adjoining a polynomial-size proof of T_n gives the mathematical estimates

$$d_S(P_n) > 2^n, \quad r_S(P_n) \leq a(n + 1)^d, \quad s(P_n) + 1 \leq b(n + 1). \quad (13)$$

These estimates imply a factor-two subsequence and superpolynomial F_S . Their proving theory and box are both the chosen guarded S .

For the ordinary box, the same family has short proofs of $\Box_B P_n$. Consequently an ordinary reflection proof, or a bridge proof $\Box_B P_n \rightarrow \Box_S P_n$, would complete a direct proof after polynomial additional work. The construction therefore also supplies the mathematical bridge obstruction

$$\ell_B(\Box_B P_n \rightarrow P_n) + q_1(n) > 2^n, \quad \ell_B(\Box_B P_n \rightarrow \Box_S P_n) + q_2(n) > 2^n$$

for suitable polynomials q_1, q_2 under its stated proof-cost conventions. Identity of standard proof files does not furnish cheap internal agreement between the boxes.

Verification scope. `TaggedSyntax.lean` proves concrete syntax, substitution, guard, and expanded-trace invariants. `TaggedBounds.lean` proves the numerical consequences of estimates

such as (13). The ordinary checker, diagonal arithmetization, guard truth, and the PA proof-length estimates still have unformalized components. This is a mathematical chosen-presentation construction with partial certificates, not a complete Lean solution of part 1 or an answer about $F_{\text{PA}_{\text{bin}}}$.

5 The conditional polynomial upper-bound route

Put $R_A = (\Box A \rightarrow A)$ and suppose a proof of R_A has length k , with $|A| = n$. A formalized Löb conversion has the following form:

$$\text{proof of } R_A \rightsquigarrow \text{proof of } \Box R_A \rightsquigarrow \text{proof of } \Box A \rightsquigarrow \text{proof of } A.$$

The middle step uses the internal Löb theorem $\Box(\Box A \rightarrow A) \rightarrow \Box A$; the last step uses the original R_A .

The audit project makes the required quantitative operations explicit. Suppose, for fixed natural constants and exponents, that:

- internalizing the given proof costs at most $u(k + n + 1)^d$;
- the internal Löb instance has a proof of length at most $v(n + 1)^e$;
- MP combines two proofs with their summed lengths plus $m(|X| + |Y| + 1)$;
- boxed formulas and reflection formulas have sizes at most $b(|X| + 1)$ and $r_0(|X| + 1)$, respectively.

Define

$$W = m(b(r_0 + 3) + 2), \quad D = \max\{d, e, 1\}, \quad K = 1 + u + v + W.$$

Then the constructed proof has length at most

$$k + u(k + n + 1)^d + v(n + 1)^e + W(n + 1) \leq K(k + n + 1)^D. \quad (14)$$

If $d \leq 1$, the sharper bound is

$$(u + 1)k + (u + v + W)(n + 1)^{\max\{e, 1\}}. \quad (15)$$

The logical internal Löb theorem is also derived from explicit HBL and fixed-point inputs. For a fixed point $G \leftrightarrow (\Box G \rightarrow A)$, the standard derivation gives $\Box G \rightarrow \Box A$, then $R_A \rightarrow G$, then $\Box R_A \rightarrow \Box G \rightarrow \Box A$. This logical proof alone supplies no polynomial character bound for its instances in the actual checker.

The first-pass logarithmic-reflection/linear-hardness estimates would rule out every polynomial overhead bound: along $n = 2^j$, their upper budgets grow polynomially in j while the direct lower bound is exponential. The audit therefore proves that the first-pass separation hypotheses and a full polynomial-conversion interface cannot both hold for the same presentation. This identifies an incompatibility of assumptions, not a contradiction in PA.

Verification scope. `AuditPass.lean` constructs the conversion and bounds its cost from the explicit `QuantitativeLobTools` interface. It proves the incompatibility just described. There is no instance of `QuantitativeLobTools` for the complete actual PA-bin checker. In particular, the first bullet above remains a substantive obligation. Even (15) generally has coefficient $u + 1$ on k , and does not settle the near-unit question in part 1.

6 Raw-string quotation and compact arithmetic traces

The quotation project addresses the agenda’s arbitrary-string abbreviation grammar. Its input is a well-scoped parsed grammar: each definition is a list of literal alphabet symbols and references to earlier definitions. Fragments may cross term or formula boundaries. For n definitions, set

$$M = n + \text{the total number of atoms in all right-hand sides.}$$

Alphabet symbols are digits below a fixed base $b \geq 2$. For an expanded word w , its summary is

$$L_w = |w|, \quad S_w = b^{|w|}, \quad V_w = \text{raw}_b(w).$$

The marked base- b string code is $S_w + V_w$, which retains leading zeros and distinguishes lengths.

6.1 A compiler that avoids printing the expanded numeral

For concatenation, the summaries obey

$$L_{xy} = L_x + L_y, \quad S_{xy} = S_x S_y, \quad V_{xy} = V_x S_y + V_y. \quad (16)$$

The empty word has summary $(0, 1, 0)$; a literal digit a has summary $(1, b, a)$. The compiler uses three arithmetic registers per source definition and generates terms containing only binary numerals, addition, multiplication, and references to previous registers. Thus $S_w = b^{|w|}$ is a semantic identity; an exponentiation symbol or the full huge numeral is not printed.

Lean proves, by induction through the grammar, that the generated terms have the three intended values. It also proves identifier framing and injectivity, freshness, and serialized character bounds. With

$$Q_b(M) = 4(b + 4)M^2 + 3M,$$

the generated definitions use at most

$$(6M + 20)Q_b(M) \quad (17)$$

characters. This is a bound in the parsed grammar mass, not expanded length.

6.2 Local equality certificates

The project adds ordinary closed-term equality proofs for the generated definitions. After a definition $u := t$, the expanded equality $u = t$ is $t = t$. A shared proof of $\forall x x = x$, its universal-instantiation axiom, and MP prove the local equality. A restricted Enderton checker verifies the required axiom cases and MP records, with their actual serialization.

The resulting complete local certificate has at most

$$20 + (40M + 100)Q_b(M) \quad (18)$$

characters. The theorem substituting a source-file budget k explicitly requires $M \leq k$. The complete source lexer and its connection to the agenda’s file coding have not been verified.

These local equalities express the definitions’ reflexive correctness after expansion. They do not yet assert the original file’s acceptance by the fixed PA-bin Bew predicate.

6.3 A single coded witness for the arithmetic trace

The next milestone packages all summaries into one natural number, rather than leaving an external list of semantic values. Use the injective pairing function

$$\pi(a, b) = (a + b)^2 + a + 1.$$

For $a + b = z$, its values occupy the interval $[z^2 + 1, z^2 + z + 1]$; these intervals are disjoint, and the value determines a within the interval. A bounded-search decoder and its inverse property are proved in Lean. Encode a triple by $\pi(L, \pi(S, V))$, and a list by zero for nil and pairing for cons.

The arithmetic-trace checker validates all instructions against decoded register values. Lean proves soundness, completeness for the intended trace, and rejection of missing entries. It may accept irrelevant extra entries; soundness identifies every register required by the program. A further compiler builds a compact arithmetic term for the entire encoded trace, and proves both acceptance and correct lookup of every source summary.

For n source definitions, trace packing adds at most

$$(8n + 20)(112n) \tag{19}$$

characters. Since $n \leq M$, the total bound is

$$20 + (40M + 100)Q_b(M) + (8M + 20)(112M), \tag{20}$$

still cubic for fixed b . The bounded-search decoder is not claimed to run in polynomial time; its huge input value may itself be represented compactly.

6.4 Checked examples and their interpretation

A doubling grammar with 64 doublings has mass 194 and expands to a word of length 2^{64} . The generated local certificate has 49,287 characters, and the trace-packing extension has 55,677, for a total of 104,964. These are generated syntax sizes, not minimum PA proof lengths. The large code is not evaluated by the demo.

A small boundary-crossing example expands to 13 characters, has mass 14, and has a complete generated bundle of 5,443 characters. Independent Python diagnostics check its printed local file and numeric trace round trip; those diagnostics are tests, not Lean kernel proofs of the full PA-bin parser.

Verification scope. This milestone verifies the parsed-grammar compiler, written-size bounds, local restricted proofs, concrete arithmetic encoding, and typed trace checker. It does not yet give a literal PA formula for the expansion relation, a polynomial-size PA proof of trace correctness, full Enderton axiom checking, or agreement with the actual fixed Bew. The remaining internal PA proof-cost step cannot be replaced by the fact that Lean proves semantic correctness.

7 Proof assembly and the finite-diagonal obstruction

The next result explains why short ordinary proofs of provability, by themselves, work against the desired witness. It accounts for the written cost of references when appending a plain proof suffix to a compressed prefix.

7.1 A concrete suffix with charged binary references

Suppose a reflection proof has written length r , the target has size s , and an independent *plain* proof of $\Box P$ has length b_0 . Define

$$L(x) = \lfloor \log_2(x+2) \rfloor + 1, \quad a = (b_0 + s + 7)(2L(b_0 + 1) + 5).$$

The suffix construction relocates the plain proof's line references and appends the final MP step using the reflection conclusion. Plainness avoids collisions between the suffix's abbreviation definitions and the prefix's names. With the structured-file validity interface, Lean proves validity and the serialized bound

$$d \leq r + aL(r). \quad (21)$$

The prefix is represented through its expanded formula history; axioms, implication, and generalization are parameterized. Thus the record encoding and reference cost are concrete, but this is not a complete PA-bin parser.

Proposition 7.1 (Necessary cost of separation). *Let $q \geq 1$ and $t \geq 0$. If (21) holds and $(q+1)r + qt \leq qd$, then*

$$r + qt \leq qaL(r), \quad (22)$$

$$r \leq 8(qa)^2 + 1, \quad (23)$$

$$d \leq (a+1)(8(qa)^2 + 4). \quad (24)$$

Proof. Substitute (21) into the separation inequality and cancel qr to obtain (22). The elementary estimate $L(r)^2 \leq 4(r+2)$, together with $r \leq qaL(r)$, gives $r^2 \leq 4(qa)^2(r+2)$. If $qa = 0$, then $r = 0$. Otherwise, for $r \geq 2$ we have $r+2 \leq 2r$, hence $r \leq 8(qa)^2$; the small cases are absorbed by the extra 1 in (23). Finally $L(r) \leq r+3$ gives $d \leq r + a(r+3) \leq (a+1)(r+3)$, proving (24). $\square$

Corollary 7.2 (An eventual exclusion threshold). *Suppose a family satisfies*

$$d_n > 2^n, \quad d_n \leq r_n + a_n L(r_n), \quad a_n \leq A(n+1)^e.$$

For fixed $q \geq 1$, set

$$C = (A+1)(8(qA)^2 + 4).$$

Then factor $1 + 1/q$ separation, including any nonnegative toll, is impossible at every index

$$n > 8(C + 3e)^2 + 1. \quad (25)$$

Proof. If separation held, (24) and the bound on a_n would give $d_n \leq C(n+1)^{3e}$. The elementary growth estimate formalized in `polynomial_lt_exponential` gives $C(n+1)^{3e} < 2^n$ under (25), a contradiction. The constants are deliberately coarse; the result excludes every sufficiently late index, not just some subsequence. $\square$

7.2 Why the original finite Gödel family triggers the issue

For the ordinary proof predicate, let

$$B_n = \Box_{2^n} G_n, \quad \text{PA} \vdash G_n \leftrightarrow \neg B_n.$$

Inside PA, the two cases already imply $\Box G_n$:

1. $B_n \rightarrow \Box G_n$ by forgetting the length bound.

2. $\neg B_n \rightarrow \Box \neg B_n$ by formalized Σ_1 completeness for a Σ_1 presentation of the decidable bounded-search condition.
3. Necessitating $\neg B_n \rightarrow G_n$ and applying distribution gives $\Box \neg B_n \rightarrow \Box G_n$.

No consistency assumption is needed for this internal argument. Consistency is used externally to prove the hardness of direct proofs. With uniform arithmetization, this mechanism supplies a uniform theorem asserting the provability of the instances. Short specializations of that theorem are the source of the proposed polynomial plain proofs of $\Box G_n$.

Completing those polynomial proof bounds for the fixed PA-bin representation would activate Corollary 7.2; it would not finish a positive part 1 witness using this family. The Lean theorem `finite_diagonal_box` checks the logical implication with its three object-theory inputs explicit. It does not formalize the PA diagonal lemma or infer one PA theorem with an object-level universal quantifier from a metatheoretic family of instance proofs.

7.3 A stronger-consistency family and the remaining gap

Consider instead

$$T = \text{PA} + \text{Con}(\text{PA}), \quad Q_n = \text{Con}(T) \upharpoonright n,$$

using a fixed ordinary checker. If T is consistent, each standard finite consistency instance Q_n has a PA proof by finite checking. However,

$$\text{PA} \not\vdash \forall n \Box_{\text{PA}} Q_n. \quad (26)$$

Here the box uses the usual represented numeral-instance function.

To prove (26), suppose PA proved the displayed universal statement. Inside T , decidable completeness gives $\neg Q_n \rightarrow \Box_{\text{PA}} \neg Q_n$. Together with $\Box_{\text{PA}} Q_n$, this would imply $\Box_{\text{PA}} \perp$, contrary to T 's axiom $\text{Con}(\text{PA})$. Thus T would prove $\forall n Q_n$, namely $\text{Con}(T)$, contradicting the second incompleteness theorem for consistent T .

This excludes the particular uniform theorem that powered the first family; it does not exclude all externally obtained short provability proofs. A suitable short PA proof of $\Box_{\text{PA}} Q_n \rightarrow Q_n$ is still missing, as is the matching lower bound for the exact compressed calculus. A reflection proof in T about the PA box would mix the two theories and would not answer the question. The slow-consistency variant does not automatically repair the route: Freund and Pakhomov obtain uniform PA-provability of the corresponding finite instances in their setting [6, Corollary 2.4; Proposition 3.7].

7.4 Why no universal cheap-box shortcut is available

Under Σ_1 soundness of ordinary PA, there is no total computable function g bounding $\ell_{\text{PA}}(\Box P)$ by $g(|P|)$ for every PA theorem P . Otherwise, on input P , compute the bound and inspect all PA files up to that length for a proof of $\Box P$. If P is a theorem, the bound makes the search succeed; if the search succeeds, Σ_1 soundness of the proof predicate makes P provable. This would decide PA theoremhood. This argument rules out a universal computable bound, not family-specific estimates.

Verification scope. The suffix construction, inequalities, threshold, and parameterized finite-diagonal logic are Lean-verified. The arithmetical arguments about T , the uniform boxed families, and theoremhood undecidability are mathematical arguments in this report; they are not complete PA formalizations in the supplied projects.

8 What remains for ordinary PA-bin

The work has identified both a structural target and a concrete implementation target. For part 1, a positive answer requires non-sublinearity of H for the actual box, while a negative answer requires $H = o(k)$, equivalently the near-unit sublinear additive bound of Theorem 2.2. Merely showing some polynomial conversion cost chooses neither alternative.

For the polynomial route in part 2, the remaining internalization chain is:

1. Connect literal input bytes, identifier decoding, and the source Gödel code to the well-scoped raw-string grammar, with a charged bound relating grammar mass to file length.
2. Write the actual PA expansion/trace formula. Generate short PA proofs of the arithmetic pairing, sequence-access, and instruction-correctness facts needed by the compact witness. Bound the *written object proofs*, not only their Lean semantics.
3. Cover the full Enderton checker, including propositional-tautology axioms, capture-avoiding substitution, generalization, MP, and the abbreviation scope rules.
4. Prove agreement with the agenda's fixed Bew and produce the notice/internal Löb operations required by `QuantitativeLobTools`, with explicit character bounds.

These are unresolved obligations, not established lemmas to be imported on trust. The raw-string quotation work handles a significant representation obstacle, while the suffix obstruction explains why completing short-box proofs for the first hard family would eliminate that candidate rather than create the required separation.

9 Certificate ledger and local reproduction

9.1 What the cumulative archive contains

All six projects are pinned to **Lean 4.19.0**. They use the libraries bundled with Lean and need no Mathlib checkout. The archive has this structure:

```
MAIS-O11-certificates/
  verify.sh
  README.md
  SHA256SUMS
  verified-build.txt
  projects/
    MAIS011FirstPass/
    MAIS011AuditPass/
    MAIS011TaggedPass/
    MAIS011QuotationPass/
    MAIS011ObstructionPass/
    MAIS011EnvelopePass/
  research-notes/
```

The research notes preserve the detailed milestone accounts and the original slow-verifier manuscript. Read their claims with the current scope distinctions in this report. The projects retain their own original verification transcripts; `verified-build.txt` records the cumulative runner's new build.

Project	Certified content and principal declarations
FirstPass	Conditional finite diagonal contradiction, abstract Löb rule, lower-bound transfer, shortest proofs, and divergence. MAIS011.SeparationHypotheses.conditional_main.
AuditPass	Internal Löb logic and polynomial conversion from explicit quantitative operations. MAIS011.internal_lob_axiom, MAIS011.QuantitativeLobTools.uniform_budget, MAIS011.QuantitativeLobTools.linear_premise_bound, MAIS011.first_pass_and_polynomial_tools_incompatible.
TaggedPass	Concrete tagged syntax and guard preservation, expanded trace acceptance under stated inputs, and numerical gap implications. MAIS011.Tagged.shape_subst, MAIS011.Tagged.guarded_instantiation_axiom, MAIS011.Tagged.no_polynomial_bridges.
QuotationPass	Raw-string compiler, local certificates, arithmetic trace checker, compact witness, and serialized cost. In namespace MAIS011.Quotation: compile_correct, compile_certificate_characters, compiled_localCertificate_checked, compiled_trace_sound, packedWitness_checked, compiled_bundle_characters.
ObstructionPass	Valid serialized suffix and explicit cost; separation obstruction and eventual threshold; abstract finite-diagonal logic. In namespace MAIS011.Obstruction: Suffix.appendBoxSuffix_normalized, separation_reflection_bound, no_separation_after, finite_diagonal_box.
EnvelopePass	Exact negative condition, affine bounds, and arbitrary factor consequence. In namespace MAIS011.Envelope.Lengths: noSpeedup_iff_sublinear_envelope, noSpeedup_iff_sublinear_converter, noSpeedup_near_unit_affine, noSpeedup_overhead_affine, not_linear_overhead_arbitrary_factor. Finite files: MAIS011.Envelope.FiniteFiles.toLengths.

9.2 Run the certificates on a Mac

Download `MAIS-011-certificates.zip` to Downloads. The following uses a fresh temporary folder, so it also works when repeating the verification:

```
mais_check_dir="$(mktemp -d "${TMPDIR:-/tmp}/mais-o11.XXXXXX")"
unzip -q "$HOME/Downloads/MAIS-011-certificates.zip" -d "$mais_check_dir"
cd "$mais_check_dir/MAIS-011-certificates"
bash verify.sh
```

The runner locates `lake`, including the usual `$HOME/.elan/bin/lake` fallback, and rebuilds every project. It then runs each `Audit.lean` with warnings treated as errors. The toolchain file selects Lean 4.19.0; Elan may download that release if it is not already installed. The runner prints a final success message only if all six projects pass. It rebuilds the proof terms; it does not just print a saved transcript.

To recheck only the latest result from the archive root:

```
cd projects/MAIS011EnvelopePass
lake build
lake env lean -DwarningAsError=true Audit.lean
```

The quotation demo is optional and is not needed for the theorem checks:

```
cd projects/MAISO11QuotationPass
lake exe quotationdemo
```

Run that last block from the archive root, rather than from inside the envelope project. The independent Python diagnostics described in the quotation project’s README are separate tests of printed examples.

9.3 How to interpret the audit

The audited theorem dependencies are contained in Lean’s standard logical axioms `propext`, `Quot.sound`, and `Classical.choice`; some theorems need fewer or none. The supplied Lean source has no admitted proofs, custom axiom declarations, `native_decide` proofs, or unsafe declarations.

An axiom report is *not* a list of a theorem’s hypotheses. A conditional theorem may have no nonstandard axioms while still taking the desired arithmetization or proof-cost estimate as a parameter. Read its printed type and the interface definitions. Moreover, lengths in these theorems refer to the specified object certificates; they do not count Lean source characters, tactic steps, or compiled `.olean` bytes.

9.4 Compile this report

The TeX source is self-contained, with its bibliography included. A standard TeX Live or MacTeX installation can produce the matching PDF using:

```
pdflatex -halt-on-error MAIS-011-progress-report.tex
pdflatex -halt-on-error MAIS-011-progress-report.tex
```

Two passes resolve the table of contents and cross-references. The PDF and TeX contain the same consolidated account; the companion archive supplies the executable certificates and detailed source notes.

References

- [1] MAIS project. *Does Löb’s rule shorten proofs?* (MAIS-O11), *MAIS-A1*, Sections 2–3 and Question 3.6, and MAIS-O10. Uploaded source documents used in this development; copies are included in the research-notes directory. <https://github.com/lionellelevine/MAIS>.
- [2] Herbert B. Enderton. *A Mathematical Introduction to Logic*, second edition. Academic Press, 2001. Section 2.4.
- [3] Andrew Critch. A parametric, resource-bounded generalization of Löb’s theorem, and a robust cooperation criterion for open-source game theory. *Journal of Symbolic Logic* 84(4), 1368–1381, 2019. <https://arxiv.org/abs/1602.04184>.
- [4] Pavel Pudlák. The lengths of proofs. In Samuel R. Buss, editor, *Handbook of Proof Theory*, pp. 547–637. Elsevier, 1998. <https://users.math.cas.cz/~pudlak/length.pdf>.
- [5] Pavel Pudlák. Incompleteness in the finite domain. *Bulletin of Symbolic Logic* 23(4), 405–441, 2017. <https://arxiv.org/abs/1601.01487>.
- [6] Anton Freund and Fedor Pakhomov. Short proofs for slow consistency. *Notre Dame Journal of Formal Logic* 61(1), 31–49, 2020. <https://arxiv.org/abs/1712.03251>.